```
var
    datacs:TfileStream;                    //定义 datacs 为文件流对象
    datauncs:TdecompressionStream;  //定义 datauncs 为解压缩对象
    buffer:Pchar                           //定义 buffer 为指向字符的指针
    count:integer;                         //定义 count 为整型变量
    databmp:TbitMap;                       //定义 databmp 为二进制位图对象
.....
//建立二进制位图
databmp:=TbitMap.Create;
//以文件流的只读方式打开 cpacs.bmp 的压缩格式文件
datacs:=TfileStream.Create（'capcs.bmp',fmOpenRead）;
//从被压缩的图像流中读出原始图像的尺寸，并分配内存
datacs.ReadBuffer（count,sizeof（count））;
GetMem（buffer,count）;
datams:=Tmemorystream.Create;
//解压缩 datacs 文件流
datauncs:=TdecompressionStream.Create（datacs）;
//解压后存入 Buffer 内存块中
datauncs:=ReadBuffer（buffer^,count）;
//将原始图像流保存至 datams 流中
datams.WriteBuffer（buffer^,count）;
//复位流指针
datams.Position:=0;
//从 datams 流中载入原始图像流
databmp.LoadFromStream（datams）;
freemem（buffer）;
datams.Free;
datauncs.Free;
//将原始图像流还原为 uncap.bmp 文件
databmp.SaveToFile（'uncap.bmp'）;
```

（6）图像的显示

图像显示部分比较简单，代码如下：

```
image1.picture.bitmap.loadfromstream（databmp）;
```

当一帧图像传输完成后，客户端向服务端发出一个消息，服务端就继续采集下一帧图像发送给客户端。

7.2 网络仿真实验

7.2.1 网络仿真器 NS2 的安装与简析

网络仿真是网络设计的重要环节，也是网络性能分析的关键。网络设计、模拟和性能分析系统的研究是一个非常有意义的课题，它使得设计者在设计阶段就能对所设计的网络的性能作出准确的预测，合理修改其设计，选择符合要求的性价比较优的方案。进行计算机仿真，

利用仿真中间参数分析系统性能是分析某一计算机网络系统性能的常用方法。

目前，采用的网络仿真方法主要有三种方式：一是通过借助大型的商用网络仿真软件进行模拟，如 OPNET；二是从有关网站获得实际的运行数据进行分析和模拟；三是使用一些免费的仿真工具进行模拟仿真，如采用 Lawrece Berkeley National Laboratory 开发的网络仿真器 NS2。商用仿真软件虽然可靠性好，但价格比较昂贵，而且扩展新的模块比较复杂，因此越来越多的研究机构选用 NS2 网络仿真器。NS2 仿真器免费提供源代码，而且可以比较方便地扩展新的仿真模块，非常适合目前的仿真研究。

NS2 为用户了提供一个与操作系统无关的平台，屏蔽掉对操作系统的实际访问，且能近乎真实地模拟网络环境，以实现在各个层次上更好地模拟网络的运行。简单地概括其功能就是：它可以在一台计算机上动态仿真一个网络的运行。

1．NS2 的运行环境及语言支持

NS2 工具包由许多模块构成，不同的组合可以在不同的操作系统上运行，如 Windows 系列、Linux、Unix、Macintosh 等。但是，整合的 ns all-in-one 套件只能在 Linux 下运行，且为了运行 NS2，还要求系统装有 C++编译器。

NS2 的一大特点，在于它采用 C++、OTcl 两种语言的架构。一方面，对于一些具体协议的仿真，比如事件的处理、数据包的转发等，要求较高的运行速度，而且一经完成就很少修改，所以 C++是一个很好的选择；另一方面，在做网络研究时，经常需要设定不同的网络环境、动态修改配置和参数，要求较高的转换速度，所以使用 OTtcl 这种直译式的语言能拥有较好的弹性。NS2 通过 tclcl 模块将出现在两种语言里的变量、对象联系起来，在两种语言特性的互补下，使得 NS2 成为兼具高效与高弹性的网络仿真软件。

2．NS2 的安装

目前，NS2 的安装主要有两种方式：一是分别安装各个所需软件；二是安装 ns all-in-one 套件。一般来说，第一次安装 NS2 最好采用后一种方式。最新版的 all-in-one 套件可从 http://www.isi.edu/nsnam/ns 下载。

以在 Linux 的/home/ns 目录下安装 ns all-in-one 套件 ns-allinone-2.26.tar.gz 为例：

```
[root@LMS ns]#tar zxvf ns-allinone-2.26.tar.gz
[root@LMS ns]#cd ns-allinone-2.26
[root@LMS ns-allinone-2.26]#./install
#若安装出现问题，可参考 http://www.isi.edu/nsnam/ns/ns-problems.html。需要特别注意的是，
#在安装过程中将出现提示，要求在 PATH、LD_LIBRARY_PATH、TCL_LIBRARY 环境变量参数
中写入提
#示的相关路径，可使用命令：export PATH=<paths>
[root@LMS ns-allinone-2.26]#cd ns-2.26
[root@LMS ns-2.26]#./validate
```

3．NS2 模块及其功能

如图 7-5 所示，粗框里为 NS2 的各个模块，框外的 script 为编写的脚本文件。NS2 将解释脚本，把结果写入到输出文件中，然后调用 Nam 或 Xgraph 显示。

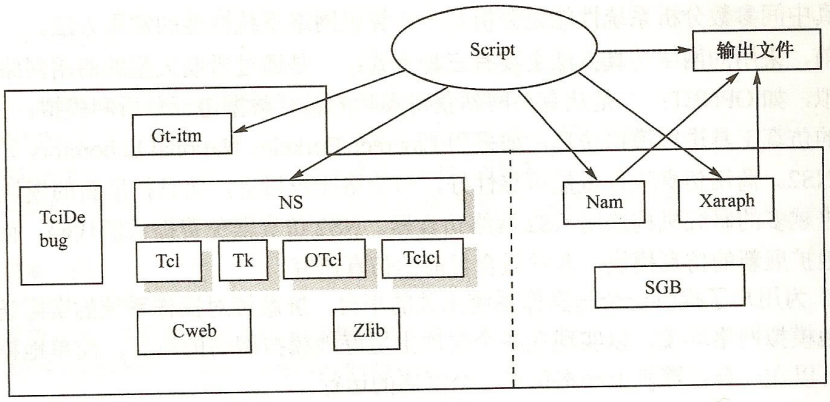


图 7-5　模块功能示意图

各个模块概述如下：

- Tcl：提供一个强有力的平台，可以生成面向多种平台的应用程序、协议、驱动程序等；与 Tk 协同工作，可用于生成 GUI 应用程序，在 PC 机、Unix 和 Macintosh 上运行；Tcl 还可用来完成与网页相关的任务，或是为应用程序提供强有力的命令语言。
- Tk：与 Tcl 协同工作的图形工具包。
- OTcl：Tcl/Tk 面向对象编程的扩展。
- Tclcl：包含 tcl/C++的接口。
- NS：NS2 主体代码，内含一个节点移动产生器、两个传输事件产生器。
- TclDebug：Tcl 调试工具包。
- Nam：与 NS2 协同工作，将 NS2 仿真过程动态地表现出来。
- Xgraph：X-Windows 应用程序，包含交互式测量和绘制，以及动画效果。
- Gt-itm：产生模拟 Internt 网络结构的拓扑图。
- SGB：图形产生器。
- Cweb：与网页相关的工具。
- Zlib：通用数据数据压缩库。

4．NS2 的使用

运行 NS2 时，可以将整个 NS2 看成一个解释器。有两种方法可以对网络事件进行仿真：

（1）命令行方式

即输入一个命令，返回一个结果，如

输入命令：ns-version

返回版本号：2.26

（2）脚本方式

即指定一个脚本文件（*.tcl 文件）让 NS2 执行，如

```
ns   <example.tcl>
```

因为脚本文件可以对网络结构、事件等做更为详细的描述，所以通常采用后一种方式。脚本采用 OTcl 语言编写，可参考~ns/otcl/doc 目录下的一份 OTcl 语言入门文件 OTcl Tutoria。

编写脚本的基本流程从创建一个仿真器的对象开始，通过这个仿真器调用各种方法生成

节点，进而构造拓扑图，对仿真的各个方面进行配置，定义事件，然后根据定义的事件，模拟整个网络活动的过程，最后调用 Nam 或 Xgraph 显示输出。

NS2 允许在脚本中定义一个输出文件（或称为跟踪文件）来记录仿真过程，文件名为“*.out”或“*.tr”，分别采用以下两种图形方式显示：

● nam 方式

编译 nam 模块，产生 nam.exe 文件，使用语句：

```
%nam <filename.out>
```

显示仿真的整个过程，如数据包的传输、链路的断开、节点的移动、丢包等；

● Xgraph 方式

编译 Xgraph 模块，产生 Xgraph.exe 文件，使用语句：

```
%xgraph “filename.tr”
```

显示仿真过程中的内部状态，如传输速率、各种峰值等。

5．NS2 的扩展

NS2 主体代码中，定义了 6 种基本类型，它们分别是：

1）Tcl 类：C++代码与 Tcl 代码之间的桥梁。

2）TclObject 类：所有仿真对象的基本类型。

3）TclClass 类：定义解释类的类层次，并允许用户实例化 TclObject。

4）TclCommand 类：封装了 C++代码和 Tcl 代码相互调用命令的方法。

5）EmbeddedTcl 类：封装了装载更高级别的内置命令的方法。

6）InstVar 类：定义了显式实现绑定机制的方法。

而 NS2 对网络实体的仿真和各种功能模块都封装在派生类中，下面按照仿真模块来介绍比较重要的派生类—仿真器（simulator）类。该类可以看成是对整个仿真器的封装，含成员类 node、link、Agent、package、LAN 等。从 NS2 外部来看，整个的仿真过程可以看成是对仿真器的操作。

Node 类主要仿真网络中的节点，其主要组分 Classifier 类实现了部分路由功能。

● Link 类仿真链路，主要组分是 Connector、Queue、Delay 类。目前 NS2 所支持的队列机制有 DropTail(FIFO)、RED、CBQ、FQ、SFQ 和 DRR。

● Agent 类主要实现对各层协议的仿真，与计时器 timmer 合作。

● Pakage 类模拟数据包，可以实现区别服务模型。

● LAN 类用于实现局域网模型。

● 对于移动节点及无线模型，NS2 则实现了所有节点都是无线节点的纯无线 LAN 和 multihop ad-hoc 网络的仿真。

● NS2 还实现了其他模型，如卫星网络模型、能量模型、无线传播模型等。

NS2 主代码说明可参考 http://www.isi.edu/nsnam/ns/ns-documentation.html。如果用户想扩展新的仿真模块，可采用以下两种方法：

（1）基本扩展

有两种方法：一种是在~tclcl/tcl-object.tcl 中加入 OTcl 代码；另一种是在目录~ns/tcl/lib

下加入新的 OTcl 脚本。采用后一种方法还必须使用如下语句在~ns/tcl/lib/ns-lib.tcl 中加入对脚本的引用：

```
source tcl/<filemame>
```

（2）进一步扩展

若采用上述方法对 NS2 进行扩展，运行时需要访问 OTcl 解释器、用户区。为了提高执行的速度，还需要将代码装载入 C++数据区。具体实现过程如下：

将脚本转为类 EmbeddedTcl 对象，使用如下语句可扩展 ns-lib.tcl 并创建一个 EmbeddedTcl 对象 et_ns_lib：

```
tclsh bin/tcl-expand.tcl tcl/lib/ns-lib.tcl | \
                                    ../Tcl/tcl2c++ et_ns_lib > gen/ns_tcl.cc
```

初始化时，~ns/ns_tclsh.cc 中的 Tcl_AppInit 函数调用~ns/tcl/lib/ns-lib.tcl：

```
et_ns_lib.load();
```

将脚本装载入 C++数据区。

7.2.2 网络仿真器 NS2 的配置与应用

1．实验要求

安装并运行网络仿真器 NS2，了解其功能模块及配套工具的使用，掌握利用 NS2 进行网络仿真的方法，为进一步的网络系统性能分析设计创造良好的条件。

2．实验条件

硬件：一台计算机；

软件：Linux 操作系统、NS2 工具包。

3．实验指导

（1）NS2 仿真流程

采用 NS2 进行仿真实验之前，必须先掌握本书 7.2.1 节中的相关内容。一般来说，采用 NS2 进行仿真大致可分为 3 个步骤：

- 建立 Network model：描述整个网络的拓扑、带宽等信息。
- 建立 traffic model：描述所有的网络流量或错误情况的时间、类型、或呈何种数学分布。
- 追踪分析结果：仿真完成后，可调用 nam 观察整个仿真流程，或是将 nam file 中的信息抽取出来加以分析。

下面是一些具体的实现语句，其中#号后的说明为该语句的相关注释。

1）建立 Network model。

- 建立 ns 对象

```
set ns [new Simulator]
```

- 建立节点

```
set n0 [$ns node]
```

● 建立连接

```
$ns duplex-link $n0 $n1 <bandwidth> <delay> <queue_type>
```

队列（queue）是保持（held）或丢弃（drop）数据包的地方，目前 NS2 所支持的队列缓冲管理机制有 drop-tail (FIFO)队列、RED 缓冲管理、公平队列（Fair Queueing,FQ）、随机公平队列（Stochastic Fair Queueing，SFQ）、DRR (Deficit Round-Robin)和基于类的队列（class-based queueing，CBQ）。

● 建立局域网

```
$ns make-lan <node_list> <bandwidth> <delay> LL Queue/DropTail MAC/802.3 Channel
```

若只想建立点对点的网络，则无需建立局域网这个步骤。

2）建立 Traffic model。

建立连接

根据不同的流量类型，可分为

TCP 连接

```
set tcp [new Agent/TCP]              #建立 TCP 代理，用以产生流量
set tcpsink [new Agent/TCPSink] #建立 TCPSink 代理，用以接收流量
$ns attach-agent $n0 $tcp          #将所建立的 tcp 配置给节点 n0
$ns attach-agent $n1 $tcpsink     #将所建立的 tcpsink 配置给节点 n1
$ns connect $tcp $tcpsink          #连接
```

UDP 连接

```
set udp [new Agent/UDP]              #建立 UDP 代理，用以产生流量
set null [new Agent/NULL]            #建立 NULL 代理，用以接收流量
$ns attach-agent $n0 $udp          #将所建立的 udp 配置给节点 n0
$ns attach-agent $n1 $null         #将所建立的 null 配置给节点 n1
$ns connect $udp $null               #连接
```

● 产生流量

对应 TCP 连接，产生 FTP(or Telnet)流量：

```
set ftp [new Application/FTP]
$ftp attach-agent $tcp
```

对应 UDP 连接，产生 CBR(or Exponential,Pareto)流量：

```
set src [new Application/Traffic/CBR]
$src attach-agent $udp
```

● 建立排程

```
$ns at <time> <event>              #<event>为任何合法的 ns/tcl 命令
$ns run
```

3）追踪分析结果。

```
$ns namtrace-all [open test.nam w]
```

其中 test.nam 为记录仿真过程的输出文件，内容包含网络的拓扑(node、link、 queues、...)，以及所有数据包的信息。用户可以使用该文件中的有关数据包的信息来作一些追踪分析。

（2）NS2 仿真示例—nam 方式

下面给出一个具体的 NS2 仿真示例，以便可以更快更清楚地了解网络仿真的实际操作流程。例如，在文本编辑器中写入 OTcl 脚本，脚本内容如下：

```
#建立一个 ns 对象
  set ns [new Simulator]

#设定第一条数据流在 nam 中用蓝色表示，第二条数据流用红色表示，主要是为了便于观察被传送
#和丢弃的数据包是属于哪个数据流，其中参数 1、2 为数据流所对应的 flow id（流量标识符）
  $ns color 1 Blue
  $ns color 2 Red

#打开跟踪文件 out.nam，其中 nf 为文件句柄
  set nf [open out.nam w]
  $ns namtrace-all $nf

#定义结束过程
  proc finish {} {
        global ns nf
        $ns flush-trace
  #关闭跟踪文件
        close $nf
  #调用 nam
        exec nam out.nam &
        exit 0
  }

#建立四个节点：n0、n1、n2、n3
  set n0 [$ns node]
  set n1 [$ns node]
  set n2 [$ns node]
  set n3 [$ns node]

#建立节点间的链路 Link，设置带宽 1Mbit/s，时延 10ms，队列 DropTail，即先进先出（FIFO）
  $ns duplex-link $n0 $n2 1Mb 10ms DropTail
  $ns duplex-link $n1 $n2 1Mb 10ms DropTail
  $ns duplex-link $n3 $n2 1Mb 10ms DropTail

#设定节点在拓扑图中的位置，使得整个图看起来布局更为合理
  $ns duplex-link-op $n0 $n2 orient right-down
  $ns duplex-link-op $n1 $n2 orient right-up
  $ns duplex-link-op $n2 $n3 orient right
```

```
#设定 nam 中 queue 的位置，便于监测节点 n2 和 n3 之间链路的队列
  $ns duplex-link-op $n2 $n3 queuePos 0.5

#为节点 n0 建立一个 UDP 代理 udp0，用于产生流量，并设定 flow id（流量标识符）为 1
  set udp0 [new Agent/UDP]
  $udp0 set class_ 1
  $ns attach-agent $n0 $udp0

#设置 udp0 产生 CBR 流量，每隔 0.005s 发送大小为 500B 的数据包
  set cbr0 [new Application/Traffic/CBR]
  $cbr0 set packetSize_ 500
  $cbr0 set interval_ 0.005
  $cbr0 attach-agent $udp0

#为节点 n1 建立一个 UDP 代理 udp1，用于产生流量,并设定 flow id（流量标识符）为 2
  set udp1 [new Agent/UDP]
  $udp1 set class_ 2
  $ns attach-agent $n1 $udp1

#设置 udp1 产生 CBR 流量，每隔 0.005s 发送大小为 500B 的数据包
  set cbr1 [new Application/Traffic/CBR]
  $cbr1 set packetSize_ 500
  $cbr1 set interval_ 0.005
  $cbr1 attach-agent $udp1

#为节点 n3 建立一个 Null 代理 null0，用以接收流量
  set null0 [new Agent/Null]
  $ns attach-agent $n3 $null0

#分别连接流量产生代理 udp0、udp1 和流量接收代理 null0
  $ns connect $udp0 $null0
  $ns connect $udp1 $null0

#排程，设置 cbr0 在仿真 0.5 s 后开始传送数据，4.5 s 后停止传送数据；设置 cbr1 在仿真 1.0s
#后开始传送数据，4.0 s 后停止传送数据
  $ns at 0.5 "$cbr0 start"
  $ns at 1.0 "$cbr1 start"
  $ns at 4.0 "$cbr1 stop"
  $ns at 4.5 "$cbr0 stop"

#仿真 5 s 后，调用结束过程
  $ns at 5.0 "finish"

#执行仿真
  $ns run
```

保存脚本文件为 example1.tcl，运行如下命令，弹出如图 7-6 所示的 nam 窗口：

```
ns example1.tcl
```

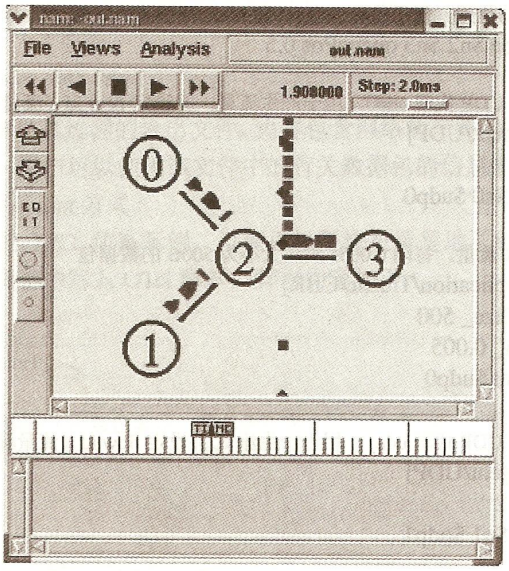


图 7-6　nam 输出示例

点击窗口上的“PLAY”按钮，将动态显示仿真过程：仿真 0.5s 后，节点 n0 开始向节点 n2 传送数据包，该数据流以蓝色动态显示，此时可以拖动窗口右上方的“Step”滑尺以减缓 nam 的速度；仿真 1.0s 后，节点 n1 开始向节点 n2 传送数据包，以红色动态显示；经过一段时间后，可观察到链路队列上的数据包如何被丢弃，因为脚本采用的是 FIFO 队列机制，所以两种颜色的数据包并没有被“公平”地丢弃，所丢弃的都是蓝色的数据包，这可以通过修改链路配置的队列机制来提高其公平性。

（3）NS2 仿真示例—Xgraph 方式

由上面的仿真示例可以看出，采用 nam 方式显示的是整个仿真的动态过程，如数据包的传输、链路的断开、丢包等；若想显示仿真过程中的内部状态，如传输速率、各种峰值等，可采用 NS2 提供的另外一种方式—Xgraph 方式。

下面给出另一个采用 Xgraph 显示方式的 NS2 仿真示例。其网络拓扑结构如图 7-7 所示，链路 Link 都是带宽 1Mbit/s、时延 100ms、采用 FIFO 队列，UDP 连接从节点 n0、n1、n2 到节点 n4，对应的 OTcl 脚本如后。

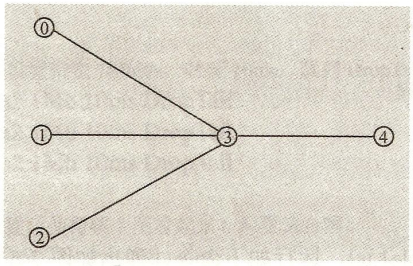


图 7-7 网络拓扑结构

在文本编辑器中写入 OTcl 脚本，脚本内容如下：

```
#建立一个 ns 对象
  set ns [new Simulator]

#打开输出文件
  set f0 [open out0.tr w]
  set f1 [open out1.tr w]
  set f2 [open out2.tr w]

#for 循环，建立 n0～n4 5 个节点
  for {set i 0} {$i < 5} {incr i}
        set n($i) [$ns node]
  }

#建立链路 Link
  $ns duplex-link $n0 $n3 1Mb 100ms DropTail
  $ns duplex-link $n1 $n3 1Mb 100ms DropTail
  $ns duplex-link $n2 $n3 1Mb 100ms DropTail
  $ns duplex-link $n3 $n4 1Mb 100ms DropTail

#定义结束过程
  proc finish {} {
      global f0 f1 f2
    close $f0
    close $f1
    close $f2
#调用 Xgraph 显示仿真结果
    exec xgraph out0.tr out1.tr out2.tr -geometry 800x400 &
      exit 0
  }

#定义过程，为节点 node 和流量接收代理 sink 建立 UDP 连接，产生 Expoo 流量，并设置流量的
#size(packetSize，数据包大小)、burst （burst_time，突发时间）、idle（idle_time，空闲时间）
#和 rate（burst peak rate，峰率）等
  proc attach-expoo-traffic { node sink size burst idle rate } {
    set ns [Simulator instance]
      #为节点 node 建立一个 UDP 代理 source，用于产生流量
    set source [new Agent/UDP]
    $ns attach-agent $node $source
      #产生 Expoo 流量
    set traffic [new Application/Traffic/Exponential]
      #设置 Expoo 流量的 packetSize(数据包大小)、burst_time(突发时间)、idle_time(空闲时间)
        #和 burst peak rate（峰率）
    $traffic set packetSize_ $size
      $traffic set burst_time_ $burst
      $traffic set idle_time_ $idle
```

```
    $traffic set rate_ $rate
        $traffic attach-agent $source
    #连接流量产生代理 source 和流量接收代理 sink
        $ns connect $source $sink
    return $traffic
    }

      #定义记录过程，周期性地记录 3 个流量接收代理 sink0、sink1、sink2 所接收的数据流带宽，并
分别写入 3 个文件 f0、f1、f2 中
   proc record {} {
        global sink0 sink1 sink2 f0 f1 f2
    set ns [Simulator instance]
       #定义过程再次被调用的时间，即调用周期
        set time 0.5
 #获取代理已接收的字节数，即变量 bytes 的值
        set bw0 [$sink0 set bytes_]
        set bw1 [$sink1 set bytes_]
        set bw2 [$sink2 set bytes_]
 #获取当前时间
        set now [$ns now]
 #计算带宽(单位:Mbit/s)，并将结果写入文件
        puts $f0 "$now [expr $bw0/$time*8/1000000]"
        puts $f1 "$now [expr $bw1/$time*8/1000000]"
        puts $f2 "$now [expr $bw2/$time*8/1000000]"
 #重置接收代理中变量 bytes 的值
        $sink0 set bytes_ 0
        $sink1 set bytes_ 0
        $sink2 set bytes_ 0
 #重新排程
        $ns at [expr $now+$time] "record"
   }

 #为节点 n4 建立 3 个流量接收代理 sink0、sink1、sink2
   set sink0 [new Agent/LossMonitor]
   set sink1 [new Agent/LossMonitor]
   set sink2 [new Agent/LossMonitor]
   $ns attach-agent $n4 $sink0
   $ns attach-agent $n4 $sink1
   $ns attach-agent $n4 $sink2

 #调用 attach-expoo-traffic 过程，分别建立节点 n0、n1、n2 与流量接收代理 sink0、sink1、sink2
 #的 UDP 连接，产生并设置 Expoo 流量
   set source0 [attach-expoo-traffic $n0 $sink0 200 2s 1s 100k]
   set source1 [attach-expoo-traffic $n1 $sink1 200 2s 1s 200k]
   set source2 [attach-expoo-traffic $n2 $sink2 200 2s 1s 300k]

 #排程
```

```
$ns at 0.0 "record"
$ns at 10.0 "$source0 start"
$ns at 10.0 "$source1 start"
$ns at 10.0 "$source2 start"
$ns at 50.0 "$source0 stop"
$ns at 50.0 "$source1 stop"
$ns at 50.0 "$source2 stop"
$ns at 60.0 "finish"

#执行仿真
$ns run
```

保存文件为 example2.tcl，运行如下命令，弹出如图 7-8 所示的 Xgraph 窗口：

```
ns example2.tcl
```

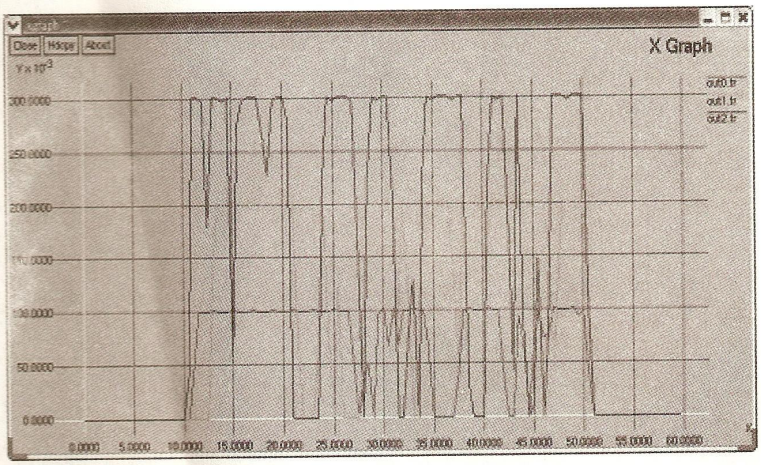


图 7-8　Xgraph 输出示例

Xgraph 显示仿真过程中的内部状态，从图中可以看出，三条数据流的峰值依次为 0.1Mbit/s，0.2Mbit/s 和 0.3Mbit/s。